

Hochschule Ravensburg-Weingarten
University of Applied Sciences



Project Report

Real-Time Digital Filter Evaluation Platform

Author: Yash Balu Sangale (29346383)

Co-Author: Neel Karia (16346352)

Study Program: M.Eng. Electrical Engineering and Embedded
Systems

Faculty of Electrical Engineering and Computer Science

Supervisor: Prof. (Dr.) Christoph Ellenrieder

Revision: 1.0

Date: August 14, 2026

Contents

1	Introduction	1
1.1	Background	1
1.2	Problem Statement	1
1.3	Objectives	1
1.4	Implemented Solution	2
2	System Design and Implementation	3
2.1	System Architecture	3
2.2	ESP32-S3 Platform Controller	4
2.3	Execution Modes	4
2.4	Test Signal Generation	5
2.5	Real-Time Sampling and Processing	5
2.6	Digital Filter Implementation	5
2.6.1	Internal Processing (ESP32-S3)	6
2.6.2	External Target Processing (Arduino Nano 33 IoT)	6
2.7	Target Operating States	7
2.8	Runtime Filter Configuration	7
2.9	Inter-Controller Communication	8
2.10	DSM Telemetry Packet	8
2.11	UDP Telemetry Transmission	9
2.12	Host-Side Visualization	9
3	Results	10
3.1	Real-Time Visualization Interface	10
3.2	Discussion	10
3.3	Limitations	11
3.4	Conclusion	11

List of Figures

2.1	Overall architecture of the real-time digital filter evaluation platform. . .	3
3.1	Final real-time graphical interface of the digital filter evaluation platform.	10

List of Tables

2.1	Runtime command interface. <code>imode</code> , <code>freq</code> , <code>amp</code> , <code>wave</code> , and <code>gcoeff</code> apply to the ESP32-S3 only; the target implements <code>fmode</code> , <code>scoeff</code> , <code>reset</code> , and <code>stop/start</code>	7
2.2	DSM packet layout. Total size is 20 B in UDP mode (with <code>seq</code>) and 18 B in USB mode (without), verified by a compile-time size assertion.	8

Chapter 1

Introduction

1.1 Background

Digital filters are a core building block of embedded signal processing, used to remove noise, shape frequency responses, and condition signals prior to further analysis or control. Filter design is conventionally a two-stage process: the filter is first evaluated offline in simulation (e.g. MATLAB), and the resulting implementation is then transferred to target hardware for verification. This separation makes it difficult to directly compare simulated and measured behaviour during development, and it does not expose implementation-specific effects such as fixed-point quantization, coefficient scaling error, execution-time variation, and sampling jitter. A real-time evaluation platform that observes the filter during on-target execution addresses this gap.

1.2 Problem Statement

Evaluating a digital filter on embedded hardware is constrained by three practical limitations:

- **Lack of real-time observability:** filter output running on a microcontroller is not visible to the developer unless it is explicitly captured and transmitted to an external system.
- **Reconfiguration overhead:** changing the filter type or coefficients on a conventional setup requires firmware modification and re-flashing, interrupting the test-and-observe cycle.
- **No unified visualization layer:** even when signal data is captured, a lightweight mechanism for synchronized multi-channel visualization with runtime-adjustable display parameters (sample window, channel visibility) is typically unavailable.

1.3 Objectives

The project implements a Real-Time Digital Filter Evaluation Platform that:

1. generates a known test signal and applies it to a digital filter under test, executed either locally or on a dedicated target microcontroller;

2. acquires the resulting signals and streams them to a host computer without manual data retrieval;
3. provides synchronized, browser-based visualization of the acquired channels with runtime-configurable display parameters.

1.4 Implemented Solution

The platform is built around an ESP32-S3, which performs signal generation and acquisition and acts as the network bridge. A separate Arduino Nano 33 IoT target microcontroller optionally executes the digital filter under test. Processed and acquired data are transmitted wirelessly via UDP to a Node.js host bridge and forwarded to a browser-based interface over WebSocket. This establishes a continuous development loop in which the signal is generated, processed on real hardware, transmitted, and visualized without manual intervention. The system architecture, implementation, and evaluation are described in Chapter 2 and Chapter 3.

Chapter 2

System Design and Implementation

The evaluation platform combines an ESP32-S3 platform controller, an Arduino Nano 33 IoT target microcontroller, real-time signal processing, an inter-controller communication link, wireless telemetry, and a host-side visualization system. The architecture supports both local execution of the filter on the ESP32-S3 and external execution on the target microcontroller, allowing the same platform to evaluate either implementation.

2.1 System Architecture

The platform consists of two embedded controllers: the ESP32-S3, which provides signal generation, control, telemetry, and communication functionality, and the Arduino Nano 33 IoT, used as the external target for filter evaluation. The overall architecture is shown in Figure 2.1.

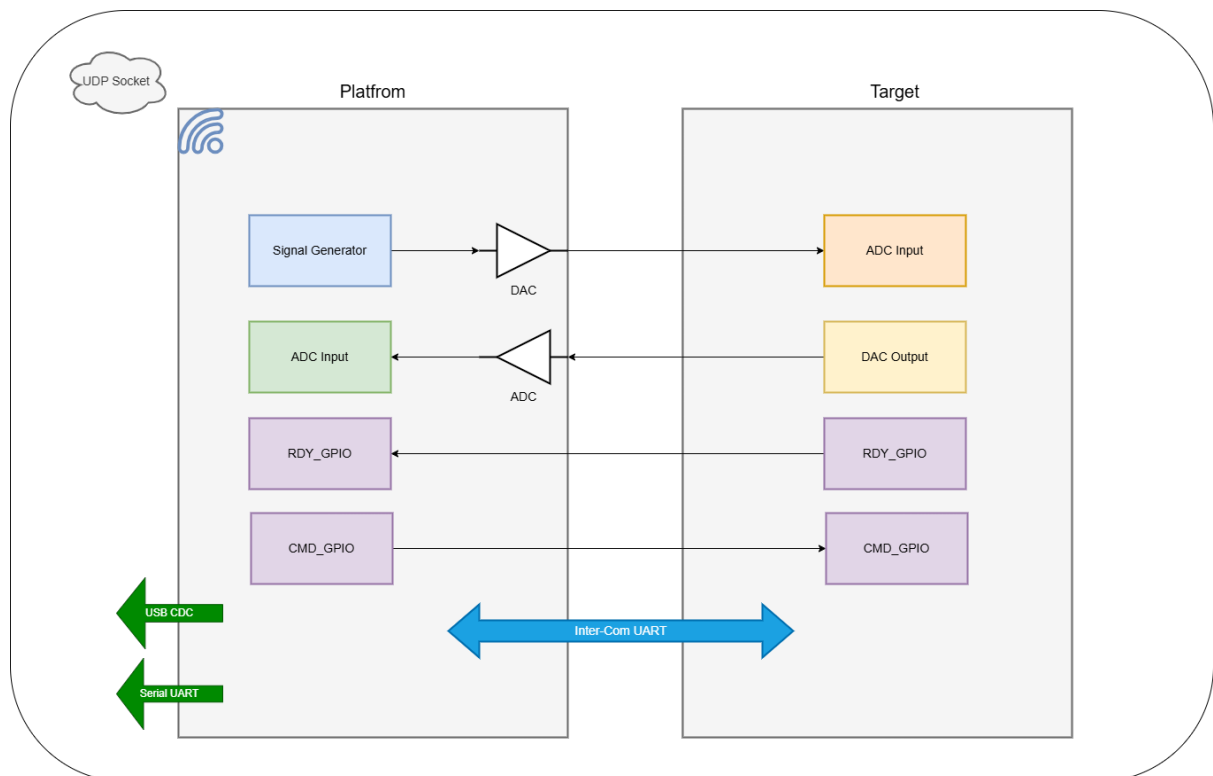


Figure 2.1: Overall architecture of the real-time digital filter evaluation platform.

The two controllers communicate over a UART-based interface with dedicated command and ready-handshake signals (Section 2.9), which allows the ESP32-S3 to control the target’s operating state and exchange configuration data at runtime. The ESP32-S3 encapsulates processed and acquired data in DSM packets (Section 2.10) and transmits them over UDP to the host application, which provides real-time visualization (Section 2.12).

2.2 ESP32-S3 Platform Controller

The ESP32-S3 is the central platform controller. Its firmware is implemented using ESP-IDF and FreeRTOS and performs:

- test-signal generation,
- acquisition of analog signals,
- execution of the digital filter in internal mode,
- communication with the external target,
- collection of target-side signals in external mode,
- runtime filter and signal configuration,
- DSM packet generation, and
- wireless UDP telemetry transmission.

The firmware is organized around a timer-driven processing loop: a hardware general-purpose timer generates the periodic sampling event, and the application task performs signal processing after receiving the timer notification. This keeps interrupt execution short and decouples the timing event from the processing workload.

2.3 Execution Modes

The platform supports two runtime-selectable execution modes.

Internal execution: the ESP32-S3 performs the complete signal-processing operation locally. On each timer event, the input is obtained from the configured signal generator or the ADC, and the selected digital filter processes the resulting sample (Section 2.6.1).

External execution: the ESP32-S3 does not perform the filter calculation. It instead coordinates the target microcontroller over the inter-controller interface, synchronizing on a GPIO handshake and sampling the target’s analog output directly rather than receiving a digital result over UART (Section 2.6.2).

The mode is switched at runtime: selecting external execution stops the internal ESP32-S3 processing timer, and reselecting internal execution restarts it. The same platform infrastructure therefore supports both local and target-side filter evaluation.

2.4 Test Signal Generation

The ESP32-S3 includes a configurable signal generator that provides deterministic test inputs, operating at the configured sampling frequency and supporting sinusoidal and impulse waveform modes. The default configuration uses a 5 kHz sampling frequency and a 100 Hz simulation frequency, with the default waveform set to impulse mode at amplitude 100 (both the ESP32-S3 and target firmware boot with this default; the frequency parameter applies to sine mode). Simulation input, frequency, amplitude, and waveform type are runtime-configurable without firmware modification.

Analog acquisition uses `ADC_UNIT_1` at 12 dB attenuation and the default ADC bit width. Channel 4 provides the primary input in both execution modes; channel 5 is used only in external mode, wired to the target’s DAC output for direct analog readback of the filtered result (Section 2.6.2).

2.5 Real-Time Sampling and Processing

The processing framework operates at a nominal sampling frequency $f_s = 5$ kHz, giving a nominal sampling period

$$T_s = \frac{1}{f_s} = \frac{1}{5000} = 200 \text{ } \mu\text{s}. \quad (2.1)$$

A hardware timer generates the periodic processing event. The timer alarm is programmed for $200 - 2 = 198 \text{ } \mu\text{s}$ rather than the nominal $200 \text{ } \mu\text{s}$, a fixed 2-tick reduction that compensates for ISR scheduling latency. The interrupt service routine performs only the minimum required operation — notifying the application task that a new sampling event is available — so that acquisition and filtering execute outside the ISR. For internal execution, the resulting sequence is:

$$\boxed{\text{Timer Event} \rightarrow \text{Input Acquisition} \rightarrow \text{Digital Filter} \rightarrow \text{Telemetry Generation}} \quad (2.2)$$

The measured execution time of the processing operation is recorded in the telemetry packet as the `dt` field, providing visibility of processing-time behaviour through the host interface.

2.6 Digital Filter Implementation

The platform supports eight filter configurations, selectable at runtime: floating-point and integer variants of direct-form IIR, FIR, second-order sections (SOS), and lattice-type filtering (LTC). The filter type is selected via the runtime command interface (Section 2.8), which passes the corresponding coefficient set to a common filter-initialization function shared by all eight configurations.

The coefficient bank implements a Bessel low-pass filter with a nominal cutoff frequency of 555 Hz at a 5 kHz sampling frequency, with both floating-point and fixed-point coefficient representations available. The default configuration is the floating-point IIR implementation.

Both the ESP32-S3 (internal mode) and the Arduino Nano 33 IoT (external mode) acquire their input via a 12-bit ADC and center the result to a signed representation using the same expression:

$$x[n] = ADC[n] - 2048. \quad (2.3)$$

The centred sample is passed to the selected filter:

$$y[n] = \text{Filter}(x[n]). \quad (2.4)$$

The two execution paths differ downstream of the filter, as described below.

2.6.1 Internal Processing (ESP32-S3)

In internal mode, the input sample (from the signal generator or ADC, Section 2.3) is centred using Eq. 2.3 and filtered using Eq. 2.4. The input and filtered samples are additionally converted to DAC-domain telemetry fields by reversing the centring offset:

$$\text{dac0} = x[n] + 2048, \quad \text{dac1} = y[n] + 2048. \quad (2.5)$$

Both signed and DAC-domain representations, together with the processing-time measurement, are stored in the telemetry structure. This path provides the local reference against which the external target implementation is evaluated.

2.6.2 External Target Processing (Arduino Nano 33 IoT)

In external mode, the ESP32-S3 does not receive the filtered sample digitally. It waits for the RDY line to fall, records the timestamp t_{low} , and samples two ADC channels: channel 4, on the shared input line ($x[n]$), and channel 5, wired to the target's analog DAC output, which yields the filtered result $y[n]$ directly as an analog readback. The ESP32-S3 then waits for RDY to return high, records t_{high} , and computes the target's per-sample processing latency:

$$dt = t_{\text{high}} - t_{\text{low}}. \quad (2.6)$$

Both channels are centred using Eq. 2.3 before insertion into the telemetry packet.

On the target itself, the Arduino Nano 33 IoT firmware implements a timer-driven acquisition and processing architecture, independent of the ESP32-S3's sampling instant: a hardware timer generates the sampling event, and the main application loop executes the processing chain

$$\boxed{\text{ADC} \rightarrow \text{Signal Centering} \rightarrow \text{Digital Filter} \rightarrow \text{DAC Scaling} \rightarrow \text{DAC}} \quad (2.7)$$

After filtering (Eq. 2.4), the result is converted from the signed 12-bit domain to the target's DAC output range, scaled while preserving the signal midpoint and clipped to the valid range:

$$\text{dac_out} = \text{clip}\left(\left\lfloor \frac{y[n] \cdot \text{DAC_MAX}}{4095} \right\rfloor + \frac{\text{DAC_MAX}}{2}, 0, \text{DAC_MAX}\right), \quad (2.8)$$

where `DAC_MAX` is the target's maximum DAC output code. This allows the target to execute the complete real-time processing chain independently of the host computer. *The target's DAC resolution (the value of `DAC_MAX`) is set by a macro not present in the supplied source; the 10-bit figure used elsewhere in the original documentation could not be independently verified against the provided files.*

2.7 Target Operating States

The target microcontroller uses command and ready signals to coordinate two mutually exclusive states:

$$\boxed{\text{Sampling State}} \quad \boxed{\text{Configuration State}} \quad (2.9)$$

In the sampling state, the target executes the timer-driven acquisition, filtering, and DAC-update cycle. When a configuration command is received from the ESP32-S3, the target disables the sampling timer, enters the configuration state, and signals its state via the ready line. After the command is processed, the target re-enables the sampling timer and resumes real-time processing. This prevents configuration operations from executing concurrently with the real-time filtering sequence. The exact handshake protocol coordinating this transition is given in Section 2.9.

2.8 Runtime Filter Configuration

Filter reconfiguration without firmware recompilation is a core design requirement (Section 1.2). The ESP32-S3 exposes a runtime command interface for filter type, input source, simulation parameters, execution mode, and other processing parameters. The full command set is implemented identically on the ESP32-S3 and the target, except where noted:

Command	Effect
<code>fmode:<iiri iirf firi firf sosi sosf ltci ltcf></code>	Select filter type/representation
<code>imode:gpio / imode:<other></code>	Select ADC input / simulation input
<code>freq:<hz></code>	Set simulation tone frequency
<code>amp:<n></code>	Set simulation amplitude
<code>wave:sin / wave:imp</code>	Select sine / impulse simulation waveform
<code>scoeff:<type></code>	Begin runtime coefficient upload for <type>
<code>gcoeff</code>	Dump active coefficients (log output)
<code>exemode:int / exemode:ext</code>	Switch internal/external execution
<code>stop / start</code>	Stop/start the sampling timer
<code>reset</code>	Restore default filter/input/freq/amp/mode
<code>ar</code>	Restart the ESP32-S3

Table 2.1: Runtime command interface. `imode`, `freq`, `amp`, `wave`, and `gcoeff` apply to the ESP32-S3 only; the target implements `fmode`, `scoeff`, `reset`, and `stop/start`.

On receipt of a filter-selection command, the corresponding configuration is applied and the filter state is reinitialized. Coefficient upload (`scoeff:<type>`) additionally

stops the sampling timer on both controllers and transfers the coefficient set as newline-terminated `w <key> <values...>` lines (e.g. `w iirf.b <21 floats>`, `w iirf.Nb <n>`), parsed identically on both firmwares. Once a complete, validated coefficient set is received, it is committed to the selected filter structure and the timer is restarted. This allows different coefficient configurations to be evaluated on the same hardware without a new firmware build.

2.9 Inter-Controller Communication

The ESP32-S3 drives CMD (GPIO4, active-low, default high) and reads RDY (GPIO5, pull-up, default high) from the target, over a secondary link (UART1, 115200 8N1). To issue a command, the ESP32-S3: waits for RDY low, drives CMD low, delays 5 ms, transmits the newline-terminated command line over UART1, then waits for RDY high before driving CMD back high:

$$\boxed{\text{RDY Low} \rightarrow \text{CMD Low} \rightarrow \text{Command Transfer} \rightarrow \text{RDY High} \rightarrow \text{CMD High}} \quad (2.10)$$

On the target, a falling CMD edge suspends the sampling timer and pulls RDY low. RDY returns high once a complete line has been received and the UART has been idle for 5 ms; the target resumes sampling once CMD returns high. This interface is not part of the high-rate signal-processing loop; it coordinates configuration and control between the two controllers.

The primary command UART (UART0, host-facing) has its TX/RX pins left as placeholder values (-1) in the supplied source and is not yet assigned to specific board pins.

2.10 DSM Telemetry Packet

After each valid processing cycle, the ESP32-S3 builds a DSM telemetry packet containing the input signal, generated signal, filter input, filter output, DAC-domain values, timing information (`dt`, Section 2.5), a sequence identifier, and an integrity field:

Field	Type	Notes
<code>sig</code>	<code>uint8</code>	Fixed signature, 0xAD
<code>n</code>	<code>uint8</code>	Packet size (bits 0–5) + rolling 2-bit seq
<code>seq</code>	<code>uint16</code>	UDP mode only
<code>adc_in</code> , <code>fgen_out</code> , <code>filt_x</code> , <code>filt_y</code> , <code>dac0</code> , <code>dac1</code> , <code>dt</code>	<code>int16 × 7</code>	Signal/timing fields
<code>crc16</code>	<code>uint16</code>	Integrity checksum

Table 2.2: DSM packet layout. Total size is 20 B in UDP mode (with `seq`) and 18 B in USB mode (without), verified by a compile-time size assertion.

The sequence identifier allows packet tracking at the receiver. Packet integrity is provided by a 16-bit checksum computed as the bitwise XOR of all 16-bit words in the packet — despite the field name `crc16`, the algorithm is an XOR checksum rather than a cyclic redundancy check. The DSM packet is the common data representation between the embedded platform and the host-side visualization system.

2.11 UDP Telemetry Transmission

Rather than transmitting each DSM packet as an individual UDP datagram, packets are accumulated into batches of `UDP_BATCH_SIZE = 100`, prefixed by a 6-byte header (4-byte batch ID + 2-byte packet count), a size verified by a compile-time assertion:

$$\boxed{\text{Sampling} \rightarrow \text{DSM Packet} \rightarrow \text{Packet Batch} \rightarrow \text{UDP} \rightarrow \text{Host}} \quad (2.11)$$

Batching reduces per-datagram transmission overhead while retaining individual sample information within each DSM packet. The sequence number and batch identifier together let the host track the received telemetry stream and detect dropped batches (Section 3.1).

2.12 Host-Side Visualization

A Node.js-based host bridge receives the binary DSM telemetry over UDP and forwards the decoded data to a browser-based interface over WebSocket:

$$\boxed{\text{ESP32-S3} \rightarrow \text{UDP} \rightarrow \text{Node.js Bridge} \rightarrow \text{WebSocket} \rightarrow \text{Browser}} \quad (2.12)$$

The browser displays discrete samples using a staircase representation rather than an interpolated continuous line, preserving the discrete-time nature of the signals and making sample-to-sample variation, jitter, quantization effects, and limit-cycle behaviour more apparent. For each displayed signal, the interface computes the minimum, maximum, average, and peak-to-peak range, and separately displays the processing-time signal `dt_us` for direct visibility of timing behaviour.

Chapter 3

Results

3.1 Real-Time Visualization Interface

Figure 3.1 shows the interface used during experimental evaluation. *The corresponding image file was not included in the source documentation (`figures/graphical_interface.png` is commented out) and could not be reproduced here.*

Figure 3.1: Final real-time graphical interface of the digital filter evaluation platform.

The interface displays the acquired signal channels — generated input, acquired signals, filter-related signals, DAC outputs, and the `dt_us` timing signal — together with the numerical statistics and communication status indicators described in Section 2.12 and Section 2.11. During evaluation, the interface was used to identify changes in signal amplitude, steady-state behaviour, timing variation, and small oscillations directly from the acquired data. The communication status indicators (received packet count, dropped-batch count, reception rate) were used to verify the integrity of the real-time data stream during testing.

3.2 Discussion

The platform combines test-signal generation, on-target processing, wireless data transport, and browser-based visualization within a single evaluation workflow, allowing the behaviour of a filter implementation to be observed directly on target hardware rather than inferred from offline simulation alone. The combination of waveform visualization, numerical signal statistics, timing information (`dt/dt_us`), and communication-integrity monitoring provides the information needed to assess both functional correctness and real-time behaviour of the embedded implementation.

The source documentation does not report quantitative evaluation results (e.g. measured filter frequency response, processing-time statistics, or packet-loss rates) obtained using the platform; only the qualitative observational capability of the interface is described.

3.3 Limitations

The target's DAC resolution constant (`DAC_MAX`, Eq. 2.8) is defined outside the supplied source files and remains unverified against the "10-bit DAC" figure used elsewhere in the original documentation. The primary command UART's pin assignment is left as a placeholder in the supplied source (Section 2.9). No quantitative performance data (frequency response accuracy, measured latency/jitter distribution, sustained UDP throughput or loss rate) is available from the source material to characterize the platform's real-time behaviour numerically.

3.4 Conclusion

The Real-Time Digital Filter Evaluation Platform addresses the limitations identified in Section 1.2: it provides real-time observability of the digital filter, eliminates the firmware re-flash step from the filter-evaluation cycle through runtime reconfiguration (Section 2.8), and provides a unified, multi-channel visualization interface (Section 2.12). The resulting workflow — test signal generation, embedded processing, wireless transmission, and real-time visualization — provides a practical basis for evaluating digital filter implementations on real hardware and for identifying implementation-specific effects not visible during offline simulation.